CardioSense AI — Cardiovascular Risk Prediction & Analytics

Name : Om Punekar
Roll No: 13 Section: A7-B1
Date: 27/10/2026

Abstract
This report builds and compares seven machine learning models that predict cardiovascular disease from eleven routine health measurements in the 70,000-record Kaggle Cardiovascular Disease dataset. Cleaning removed 1,477 duplicate or impossible records, and the remaining 68,523 were split 70/15/15 with stratification. Gradient Boosting, selected on validation ROC-AUC, reaches 73.7% test accuracy and a ROC-AUC of 0.802; the top three models differ by less than 0.3 points, and systolic blood pressure is the dominant feature. The best model is served through a responsive Streamlit web app (phone, tablet and desktop layouts) that runs locally and is deployed on Streamlit Community Cloud (link in Section 7.3).

1. Introduction
Cardiovascular disease (CVD) is a leading cause of death worldwide, and early screening depends on simple measurements such as blood pressure, cholesterol and body weight. A model that turns these measurements into a risk score can help a clinician or a patient decide who should be examined further. This project builds a machine learning system that predicts whether a patient has CVD from eleven routine health measurements and serves the prediction through a web app. The selected model, Gradient Boosting, reaches 73.7% accuracy and a ROC-AUC of 0.802 on a held-out test set of 10,279 patients. A recent systematic review reports that large datasets are scarce and that black-box models hinder clinical use (Zhou et al., 2024), which motivates a careful, reproducible comparison on a large public dataset.

1.1 Dataset
The project uses the public Kaggle Cardiovascular Disease dataset: 70,000 patient records, 11 input features and one binary target (cardio). The classes are almost perfectly balanced (35,021 healthy patients versus 34,979 with disease), so accuracy is a fair headline metric (Table 1).

Table 1. Variables of the Kaggle Cardiovascular Disease dataset

Feature	Type	Description
age	numeric	Age in days (converted to years during cleaning)
gender	categorical	1 = female, 2 = male
height, weight	numeric	Height in cm, weight in kg
ap_hi, ap_lo	numeric	Systolic and diastolic blood pressure (mmHg)
cholesterol, gluc	ordinal	1 = normal, 2 = above normal, 3 = well above normal
smoke, alco, active	binary	Smoking, alcohol intake, physical activity
cardio	binary	Target: presence of cardiovascular disease (1 = yes)

1.2 Objectives and contributions
- Clean the raw data and split it into training, validation and test sets without leakage.
- Compare seven classification models on the same split using accuracy, precision, recall, F1 and ROC-AUC.
- Explain which features drive the prediction.
- Serve the best model through a responsive Streamlit web app whose layout adapts to phone, tablet and desktop screens.

The project delivers a reproducible pipeline (preprocess.py, train_models.py), a saved best model, a Streamlit web app and one-command execution through run.py. All code and figures are in the project GitHub repository.

2. Literature Review

Seven recent papers (2024-2025) were read and compared (Table 2). Their reported accuracies range from 73% to above 97%, but the high figures come from small datasets, while the study on the same 70,000-record Kaggle data stays near 73%, which is where this project also lands.

2.1 Summary of the reviewed papers

Table 2. Comparison of the seven reviewed papers

Paper	Data	Methods	Key result	Limitation
El-Sofany et al. (2024), Scientific Reports	Cleveland + private Egyptian set: 503 records	10 classifiers, SMOTE, explainable AI	XGBoost + SMOTE: 97.57% accuracy, 98% AUC	Very small dataset; accuracy quoted three slightly different ways
Slime et al. (2024), IAENG Int. J. Comput. Sci.	Same Kaggle data as this project: 70,000 records	20 models, 8-feature and full-feature runs	XGBoost best: 73.43% test accuracy	Decision Tree: 99.99% train but 63% test (overfitting)
Liu et al. (2025), Eur. Heart J. Digital Health	Meta-analysis of health-record studies	Random forest, deep learning vs risk scores	Pooled AUC: RF 0.865, DL 0.847, risk scores 0.765	Very high heterogeneity ($I2 > 99\%$)
Deng et al. (2025), PLoS One	BRFSS 2015 (253,680) + Framingham, Z-Alizadeh Sani	LightGBM vs XGBoost, RF, CatBoost, others	BRFSS: 90.6% accuracy, AUROC 0.811	AUROC falls to 0.674 on Framingham
Kahraman (2025), Frontiers in AI	Two merged Kaggle sets: 311,710 records	10 models, GridSearchCV, SMOTE	CatBoost best: AUC 0.944, F1 0.872	Merged sources and SMOTE: not directly comparable
Zhou et al. (2024), AI Review	Review of 64 studies (2018-2023)	Deep learning; CNN in 31% of studies	Cleveland is the most used dataset	Large datasets scarce; black-box models
Kumar et al. (2025), Frontiers in AI	Review of ML for heart disease	Classic ML, deep, quantum, federated learning	Reviewed studies report up to 99.67% accuracy	Results come from other studies, not own experiments

2.2 Comparison and discussion

Dataset size explains most of the spread in accuracy. El-Sofany et al. (2024) reach 97.57% on 503 records, and Kumar et al. (2025) quote reviewed studies above 99% on the UCI Cleveland data. On the 70,000-record Kaggle data, Slime et al. (2024) report 73.43% for their best model. Large, noisier datasets give lower but more realistic numbers.

Boosted trees and random forests are consistently strong on tabular data. Liu et al. (2025) find random forest has the highest pooled AUC (0.865) across health-record studies, ahead of deep learning (0.847) and conventional risk scores (0.765). Deng et al. (2025) choose LightGBM, and Kahraman (2025) finds CatBoost best on merged Kaggle data. This supports using gradient boosting and random forest as the main candidates here.

Overfitting and external validity are the common warnings. Slime et al. (2024) show a Decision Tree with 99.99% training and 63% test accuracy, and Deng et al. (2025) show AUROC dropping from 0.811 to 0.674 on a different population. Zhou et al. (2024) add that black-box models and the lack of large datasets block clinical use. This project therefore keeps a separate validation set, limits tree depth and reports permutation feature importance.

2.3 Gap addressed by this project

Most reviewed studies report one best model or use small datasets. This project compares seven models on one clean, stratified split of a large public dataset, selects the winner on a validation set, reports the untouched test result and ships the winner as a working web app.

3. Methodology

3.1 Pipeline

The project follows a six-step pipeline in which the test set is used only once, at the end (Figure 1). Every step is a script in the repository, so the whole experiment reruns from the raw CSV with python run.py. Selecting the model on the validation set and reporting on the test set avoids choosing a model only because it happened to score well on the data used for the final report.

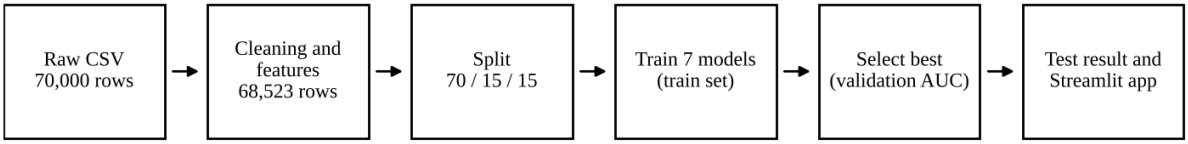


Figure 1. Experimental pipeline from raw data to the deployed app

3.2 Models and settings

Seven models cover linear, distance-based, tree-based, margin-based (SVM) and boosted approaches. Scale-sensitive models (Logistic Regression, KNN, SVM) use a StandardScaler pipeline fitted on training data only. The Decision Tree uses criterion = entropy (information gain, the idea behind ID3); scikit-learn builds CART-style trees, so it is not an exact ID3 implementation. The SVM has no probability calibration, so its ROC-AUC uses decision_function scores and it cannot be the deployed model.

Table 3. Models and their main settings

Model	**Family**	**Main settings**
Logistic Regression	Linear	Standardised inputs, max_iter 1000
KNN	Distance-based	Standardised inputs, k = 25
Decision Tree (entropy)	Tree	criterion = entropy, max_depth 6
SVM (RBF)	Margin-based	Standardised inputs, RBF kernel, C = 1, no probability calibration
Random Forest	Bagged trees	300 trees, max_depth 10
Gradient Boosting	Boosted trees	200 trees, depth 3, learning rate 0.1
HistGradientBoosting	Boosted trees (histogram)	300 iterations, learning rate 0.06

3.3 Evaluation metrics

With TP, TN, FP and FN the counts of true and false positives and negatives: Accuracy = (TP + TN) / (TP + TN + FP + FN); Precision = TP / (TP + FP); Recall = TP / (TP + FN); F1 = 2 · Precision · Recall / (Precision + Recall). ROC-AUC measures how well the predicted probability ranks sick patients above healthy ones across all thresholds, independent of the 0.5 cutoff. In a screening setting recall matters most, because a missed patient costs more than a false alarm.

3.4 Tools

Python with pandas, scikit-learn, matplotlib, seaborn and joblib for the pipeline, and Streamlit for the interface. Permutation importance on the validation set explains the best model, because it works for every model type.

3.5 Reproducibility and tuning

Every random operation (the split, the forests, the network) uses random_state = 42, so reruns give the same numbers. The settings in Table 3 were chosen by hand and not tuned by grid search, so a tuned model might score slightly higher; the comparison is therefore between sensible, untuned configurations of each model family.

4. Data Cleaning and Preprocessing

Cleaning removed 1,477 of 70,000 rows (2.1%), leaving 68,523 records that are physiologically plausible (Table 5). The raw file has no missing values, but it contains impossible entries, such as a systolic pressure of 16,020 mmHg and a height of 55 cm (Table 4).

4.1 Problems found in the raw data

Table 4. Value ranges in the raw data that revealed errors

Column	Raw minimum	Raw maximum	Problem
ap_hi (systolic BP)	-150	16,020	Negative and absurdly high values
ap_lo (diastolic BP)	-70	11,000	Negative and absurdly high values
height (cm)	55	250	Entries that are not adult heights
weight (kg)	10	200	Entries that are not adult weights
age	10,798 days	23,713 days	Stored in days, not years

4.2 Cleaning steps and their effect

Table 5. Cleaning rules and the number of rows removed by each

Step	Rule	Rows removed	Rows left
Raw data	Load with sep=";"	0	70,000
Missing values	Checked every column	0	70,000
Duplicates	Drop identical rows, ignoring id	24	69,976
Blood pressure	Keep ap_hi 70-250, ap_lo 40-200, ap_hi > ap_lo	1,329	68,647
Height, weight, BMI	Keep height 130-210 cm, weight 35-200 kg, BMI 12-60	124	68,523

The limits are wide on purpose. They remove only values that cannot occur in a living adult, so genuine hypertension (for example 200/120) stays in the data.

4.3 Feature engineering

- age_years = age in days / 365.25, rounded, because years are easier to read and explain.
- bmi = weight in kg / height in metres squared, which combines two features into one clinically meaningful measure.
- id and the original age column were dropped, since id carries no information. Scaling is done inside the model pipelines, so the scaler never sees validation or test data.

4.4 Train, validation and test split

The cleaned data was split once, stratified on the target so that every part keeps the same share of patients with disease (49.5%), with random_state = 42 for reproducibility. The training set fits the models, the validation set picks the best one, and the test set is used only for the final numbers (Table 6).

Table 6. Stratified train / validation / test split

Split	Rows	Share	Positive rate
Train	47,966	70%	49.5%
Validation	10,278	15%	49.5%
Test	10,279	15%	49.5%

5. Results and Discussion

The selected model, Gradient Boosting, classifies 73.7% of the 10,279 test patients correctly and ranks sick above healthy patients with a ROC-AUC of 0.802. Systolic blood pressure is by far the strongest signal in the data.

5.1 What the data shows

Patients with disease are older (54.9 vs 51.7 years on average), have higher systolic pressure (133.9 vs 119.6 mmHg) and a higher BMI (28.4 vs 26.4) (Figure 2). The share of patients with disease rises steeply

with cholesterol level: 43.6% at level 1, 59.7% at level 2 and 76.2% at level 3. Systolic pressure has the highest correlation with the target (0.43), followed by diastolic pressure (0.34), age (0.24) and cholesterol (0.22); smoking, alcohol, gender and height are almost uncorrelated with it (absolute value below 0.02) (Figure 3).

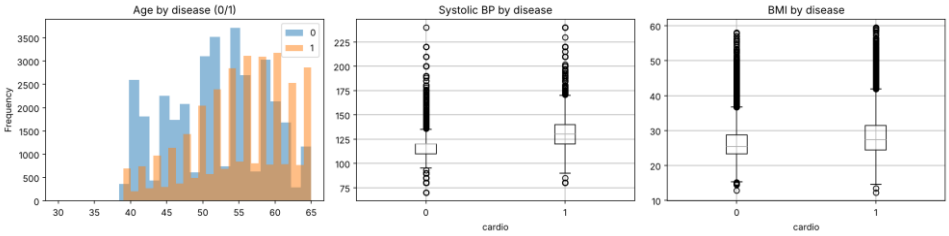


Figure 2. Age, systolic blood pressure and BMI for patients without (0) and with (1) disease

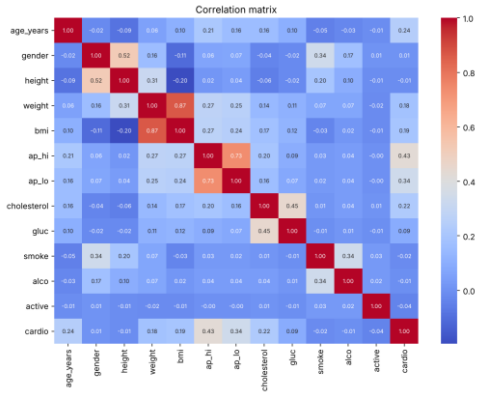


Figure 3. Correlation matrix of the cleaned data

5.2 Test-set results of the selected model

Gradient Boosting was selected because it had the highest validation ROC-AUC (0.8002). Out of 5,086 patients with disease the model finds 3,514 (recall 69.1%) and misses 1,572; out of 5,193 healthy patients it correctly clears 4,058 (specificity 78.1%) and raises 1,135 false alarms (Figure 4). Table 7 lists the metrics.

Table 7. Test-set metrics of Gradient Boosting

Metric	Test value	Meaning
Accuracy	0.7366	Share of all patients classified correctly
Precision	0.7559	Of patients flagged as sick, share truly sick
Recall (sensitivity)	0.6909	Of truly sick patients, share found
F1 score	0.7219	Balance of precision and recall
ROC-AUC	0.8022	Ranking quality over all cutoffs

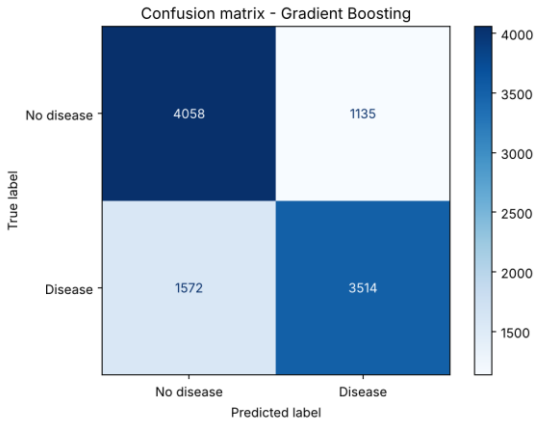


Figure 4. Confusion matrix on the test set

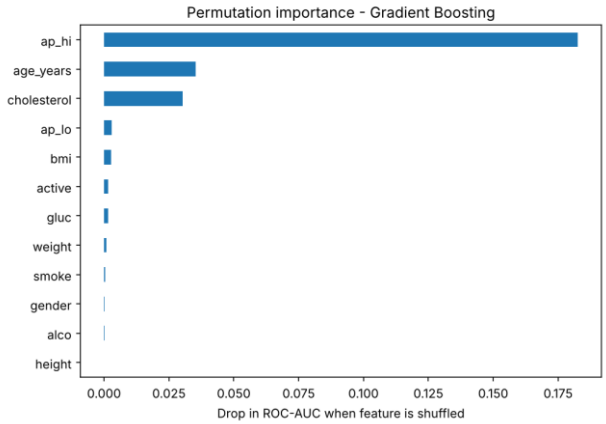


Figure 5. Permutation importance (validation set)

5.3 Precision versus recall

The default cutoff of 0.5 favours precision. For a screening tool, lowering the cutoff catches more sick patients at the price of more false alarms (Table 8). A clinic that prefers to over-refer rather than miss a patient could choose a cutoff near 0.4 and give up one point of accuracy for more than eight points of recall.

Table 8. Effect of the decision cutoff on the test set (Gradient Boosting)

Cutoff	Accuracy	Precision	Recall	Missed patients	False alarms
0.5	73.7%	75.6%	69.1%	1,572	1,135
0.4	72.7%	70.3%	77.6%	1,140	1,664
0.3	68.3%	63.0%	87.5%	633	2,620

5.4 Performance by blood-pressure group

Table 9 splits the test set by systolic pressure. Above 140 mmHg, 2,772 patients (27% of the test set) are almost all diseased, and the model labels nearly all of them as diseased (recall 99.9% to 100%), so its accuracy there simply equals the disease rate. The hard cases are the 5,605 patients (55%) with systolic pressure between 120 and 139, where accuracy is only 63% to 68%. In effect, the model behaves largely like a blood-pressure threshold, and the other features refine the decision only in the middle range.

Table 9. Test-set performance of Gradient Boosting by systolic blood-pressure group

Systolic BP (mmHg)	Patients	With disease	Accuracy	Recall
<120	1,902	23.4%	78.4%	18.2%
120-129	4,239	35.0%	68.2%	26.8%
130-139	1,366	60.5%	63.0%	85.4%
140-159	2,108	82.8%	82.8%	99.9%
>=160	664	88.1%	88.1%	100.0%

5.5 Which features matter

Permutation importance measures how much the validation ROC-AUC drops when one feature is shuffled (Figure 5). Systolic pressure dominates: shuffling it costs 0.183 AUC, five times more than age (0.035) or cholesterol (0.030). Diastolic pressure (0.003) and BMI (0.003) follow, and all other features cost less than 0.002. Diastolic pressure has a high correlation with the target but a low importance, most likely because it carries much of the same information as systolic pressure.

5.6 Discussion and limitations

- Accuracy plateau. About 73-74% is the ceiling for every model tried (Section 6), matching Slime et al. (2024). Eleven basic measurements cannot separate all patients.
- Features agree with medicine. Blood pressure, age and cholesterol are established cardiovascular risk factors, so the model learns plausible patterns rather than noise.
- Single dataset. Results come from one dataset with a random split; Deng et al. (2025) show performance can fall on another population, so external validation is needed before clinical use. About 10% of validation and test rows share a feature vector with a training row (not label leakage, but worth noting).
- Not a diagnosis. The app is an educational tool, not a medical device.

5.7 Conclusion and future work

Seven models were compared on a cleaned, stratified split of 68,523 records. Gradient Boosting, selected on validation ROC-AUC, reaches 73.7% test accuracy and a ROC-AUC of 0.802. The differences among the top three models are within the margin of error of the test set, so the choice of features and decision cutoff matters more than the choice of model. Blood pressure, age and cholesterol drive the predictions, and the model is served through a responsive web app that runs locally. Future work:

- tune hyperparameters with cross-validation and add XGBoost, LightGBM and CatBoost to the comparison;
- choose the cutoff by clinical cost (for example 0.4 for screening) and calibrate the probabilities;
- add SHAP explanations to the app, validate on an external dataset, and test the deployed app on physical phones and tablets.

6. State-of-the-Art Models Comparison

Seven models were trained on the same training set and scored on the same untouched test set. The three ensemble models (Gradient Boosting, Random Forest and HistGradientBoosting) finish within 0.001 ROC-AUC of each other, and even the simplest model, Logistic Regression, is only about 1 point of accuracy behind the best. Rows in Table 10 are ordered by validation ROC-AUC, the criterion used to pick the final model among models that output probabilities; Figures 6 and 7 show the same comparison graphically. Here "state-of-the-art" means a comparison of standard, widely used scikit-learn model families; no claim of novelty or of being the best possible model is made.

Table 10. Validation and test performance of all seven models (test set: 10,279 patients; selected model first). Training times depend on the computer; the SVM took about 138 s.

Model	Val AUC	Accuracy	Precision	Recall	F1	Test AUC	Train time (s)
Gradient Boosting	**0.8002**	**0.7366**	**0.7559**	**0.6909**	**0.7219**	**0.8022**	**11.9**
Random Forest	0.8000	0.7374	0.7640	0.6791	0.7191	0.8023	1.6
HistGradientBoosting	0.7997	0.7347	0.7556	0.6856	0.7189	0.8026	0.4
Logistic Regression	0.7925	0.7274	0.7537	0.6671	0.7078	0.7931	0.3
Decision Tree (entropy)	0.7908	0.7293	0.7514	0.6768	0.7121	0.7931	0.1
SVM (RBF)	0.7900	0.7357	0.7637	0.6744	0.7163	0.7922	137.7
KNN	0.7826	0.7251	0.7435	0.6783	0.7094	0.7842	0.2

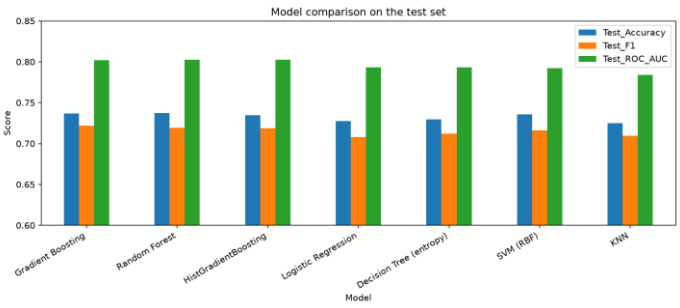


Figure 6. Test accuracy, F1 and ROC-AUC of all models

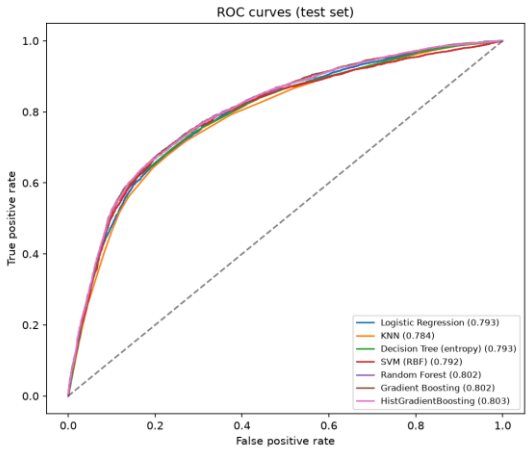


Figure 7. ROC curves of all models on the test set

6.1 Analysis

- The top three models are statistically tied. Their test accuracy ranges from 0.7347 to 0.7374. With 10,279 test patients the 95% margin of error is about 0.9 points, so a gap of up to 0.3 points does not show that one model is truly better.
- Each model leads on a different metric. Random Forest has the highest accuracy (0.7374) and precision (0.7640), Gradient Boosting the highest recall (0.6909), and HistGradientBoosting the highest test ROC-AUC (0.8026) while training in 0.4 s. Gradient Boosting was chosen because it led on the validation set, not the test set.
- Boosting and bagging beat single models. Gradient Boosting, Random Forest and HistGradientBoosting beat the single entropy Decision Tree by 0.5 to 0.8 points of accuracy.

- A simple model is nearly as good. Logistic Regression trains in 0.3 s, can be explained through its coefficients and is within 1 point of the best accuracy. KNN is the weakest (accuracy 0.7251, ROC-AUC 0.7842) and must keep all training data to predict. The RBF SVM is competitive (accuracy 0.7357) but took about 138 s to train.
- Results match the literature. Slime et al. (2024) report 73.43% test accuracy for their best model on the same dataset, close to the 73.66% obtained here.

6.2 Overfitting check

Because the literature warns about overfitting (Slime et al., 2024), Table 11 compares training and validation accuracy of every model. The gap is at most 2.2 points. Gradient Boosting generalises well, with a gap of only 0.2 points, while Random Forest (2.2 points) and KNN (2.1 points) fit the training data slightly more closely than unseen data. Limiting the Decision Tree to depth 6 keeps its gap at 0.4 points, in contrast to the 99.99% training versus 63% test accuracy reported for an unrestricted tree.

Table 11. Training versus validation accuracy (gap = train minus validation)

Model	Train accuracy	Validation accuracy	Gap (points)
Gradient Boosting	0.7397	0.7377	0.2
Random Forest	0.7563	0.7340	2.2
HistGradientBoosting	0.7425	0.7349	0.8
Logistic Regression	0.7272	0.7262	0.1
Decision Tree (entropy)	0.7336	0.7293	0.4
SVM (RBF)	0.7395	0.7348	0.5
KNN	0.7447	0.7237	2.1

6.3 Models not included and exploratory clustering

The literature favours XGBoost, LightGBM and CatBoost (Deng et al., 2025; Kahraman, 2025); they are not part of this project's code or results. An exploratory K-Means analysis (clustering.py, label not used) chose k = 3 by silhouette (0.19) and found clusters with disease rates of 36.6%, 73.5% and 45.9%; the groups overlap strongly and the analysis is not used by the predictor.

7. User Interface Development and Deployment

The selected Gradient Boosting model is served through a Streamlit web app (app/app.py) that takes eleven inputs and returns a risk probability in one click. Streamlit turns a Python script into a web page without HTML, CSS or JavaScript, so the model, the preprocessing and the interface live in one language and one repository. Because the page renders in a browser, the same code serves every screen size; the layout adapts to phone, tablet and desktop widths and was checked in browser emulation, not on physical devices.

7.1 App design

Table 12. Tabs of the web app

Tab	Purpose	Content
Predict	Main screen	Input form, "Predict risk" button, probability and risk band
Threshold analysis	Explore the cutoff	Threshold slider with live metrics, confusion matrix and curve
Model performance	Evidence for the choice of model	Comparison table, bar chart, ROC curves, confusion matrices, feature importance, leakage checks
Data insights	Context	EDA figures, correlation matrix, data-cleaning summary, exploratory K-Means clusters

The Predict tab collects age, gender, height, weight, systolic and diastolic pressure, cholesterol and glucose level (dropdowns) and smoking, alcohol and activity (checkboxes). Number inputs are limited to ranges close to those kept in cleaning, and BMI outside 12-60 is rejected. An educational-use disclaimer banner is visible on every screen size. The layout is responsive: on phones (about 375 px wide) the metric cards form a 2 x 2 grid, the four tabs wrap onto two rows and the form and result stack; tablets and desktops

show four cards in a row and a two-column form. Touch controls are at least 44 px high and no page scrolls sideways.

7.2 How a prediction is made

1. The app loads the saved model once (@st.cache_resource), so later clicks are fast.
2. It rejects input where systolic pressure is not higher than diastolic pressure, or where BMI is outside the trained range.
3. It computes BMI from height and weight exactly as in training and orders the columns as the model expects.
4. It calls predict_proba and shows the probability of disease with a progress bar.
5. It maps the probability to a band: 60% or more is high risk, 40-60% moderate, below 40% low.

Table 13 shows three example patients scored by the saved model, one in each risk band. The values are real outputs of best_model.joblib (outputs/example_predictions.csv), and the same inputs can be typed into the app to reproduce them. Figures 8 and 9 show the Predict tab in a desktop window and in tablet- and phone-sized windows (browser emulation).

Table 13. Example predictions of the saved model (local app)

Case	Inputs	BMI	Probability	Risk band
A	Man, 50 y, 165 cm, 70 kg, BP 120/80, cholesterol 1, glucose 1, non-smoker, active	25.7	23.4%	Low
B	Woman, 55 y, 160 cm, 70 kg, BP 130/85, cholesterol 1, glucose 1, non-smoker, active	27.3	50.6%	Moderate
C	Man, 60 y, 170 cm, 90 kg, BP 160/100, cholesterol 3, glucose 2, smoker, not active	31.1	87.7%	High

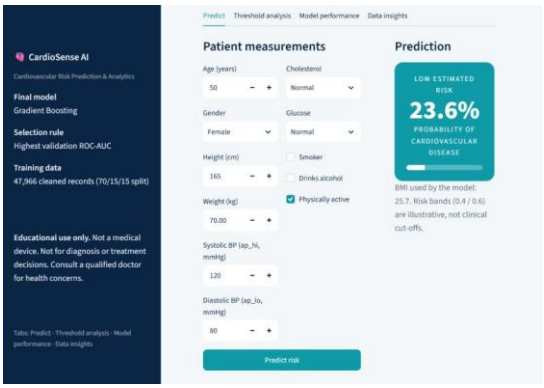


Figure 8. Desktop view (1024 px)

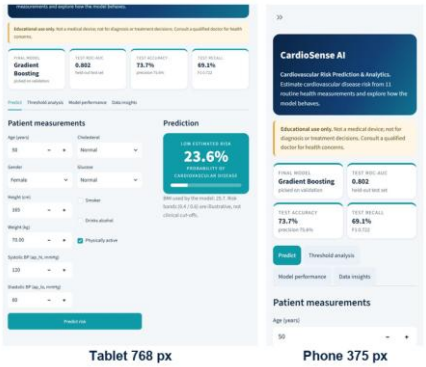


Figure 9. Tablet and phone views (emulated)

7.3 Running locally, same-Wi-Fi access and deployment

Local run: open the project folder in VS Code and run python run.py in the terminal. It installs the packages, cleans the data, trains the models and starts the app at http://localhost:8501.

Same-Wi-Fi access from a phone (local network only): start the app with python -m streamlit run app/app.py --server.address 0.0.0.0, find the computer's IPv4 address (ipconfig on Windows), connect the phone to the same Wi-Fi and open http://<computer-ip>:8501. Some hotspots and guest networks block device-to-device traffic. The app has no login, so use it only on trusted networks. This was verified only up to the server answering on the computer's own network address, not from a physical phone.

Public cloud deployment: the app is deployed on Streamlit Community Cloud from the GitHub repository (main file app/app.py). Steps:

1. Push the project folder to a public GitHub repository, including models/best_model.joblib, outputs/ and requirements.txt.
2. Open share.streamlit.io and sign in with GitHub.
3. Choose New app, select the repository and branch, and set the main file path to app/app.py.
4. Click Deploy. Streamlit installs requirements.txt (scikit-learn is pinned to 1.9.1; use Python 3.12 or newer) and publishes a public link.
5. Open the link on a laptop and a phone (only emulated checks so far; see 7.4).

Live app: https://cardiosense-risk-predictor.streamlit.app/ GitHub repository: https://github.com/Anamom-exd/CardioSense-AI

7.4 Testing and limitations of the interface

What was tested: the layout was checked in desktop-browser emulation at 375 x 812 px (phone), 768 x 1024 px (tablet) and a 1024 px desktop window on all four tabs: no horizontal page overflow, controls at least 44 px high, text at least 12 px, and tables, charts and images within the screen width. Predictions for 12 test inputs (all three risk bands) were identical before and after the layout changes and equal to the saved model's output. The live site was checked the same way (no overflow; 4 test inputs matched the local model); Streamlit Cloud's own badges can cover part of a phone screen. Not tested: physical Android or iOS devices, other browsers, and access from another device over Wi-Fi.

- The app shows an estimated probability from a model with 73.7% accuracy and must not be used as a diagnosis.
- Emulation does not replace testing on real phones, tablets and browsers; rendering may differ.
- On a free hosting tier, idle apps may sleep, so the first visit after a pause can be slow.
- The model is a static file; retraining requires rerunning train_models.py and redeploying.

8. References

Deng, L., Lu, K., & Hu, H. (2025). An interpretable LightGBM model for predicting coronary heart disease: Enhancing clinical decision-making with machine learning. PLoS ONE, 20(9), e0330377. https://doi.org/10.1371/journal.pone.0330377

El-Sofany, H., Bouallegue, B., & Abd El-Latif, Y. M. (2024). A proposed technique for predicting heart disease using machine learning algorithms and an explainable AI method. Scientific Reports, 14, 23277. https://doi.org/10.1038/s41598-024-74656-2

Kahraman, A. (2025). Machine learning techniques for improved prediction of cardiovascular diseases using integrated healthcare data. Frontiers in Artificial Intelligence, 8, 1694450. https://doi.org/10.3389/frai.2025.1694450

Kumar, R., Garg, S., Kaur, R., Johar, M. G. M., Singh, S., Menon, S. V., Kumar, P., Hadi, A. M., Hasson, S. A., & Lozanović, J. (2025). A comprehensive review of machine learning for heart disease prediction: challenges, trends, ethical considerations, and future directions. Frontiers in Artificial Intelligence, 8, 1583459. https://doi.org/10.3389/frai.2025.1583459

Liu, T., Krentz, A., Lu, L., & Curcin, V. (2025). Machine learning based prediction models for cardiovascular disease risk using electronic health records data: systematic review and meta-analysis. European Heart Journal - Digital Health, 6(1), 7-22. https://doi.org/10.1093/ehjdh/ztae080

Slime, K., Maizate, A., Hassouni, L., & Mouine, N. (2024). Toward a model to predict cardiovascular disease risk using a machine learning approach. IAENG International Journal of Computer Science, 51(5), 519-527.

Zhou, C., Dai, P., Hou, A., et al. (2024). A comprehensive review of deep learning-based models for heart disease prediction. Artificial Intelligence Review, 57, 263. https://doi.org/10.1007/s10462-024-10899-9

Ulianova, S. Cardiovascular Disease dataset (70,000 records). Kaggle. https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Streamlit documentation. https://streamlit.io